

Understanding the Evoscope Autoencoder Initialization Robustness Analysis

Overview

The purpose of the script is to determine whether independently initialized autoencoders preserve a common organization of the morphology-derived latent trajectory, even though the individual latent coordinates may differ.

The logical structure of the analysis is:

Latent coordinates $\longrightarrow$ Distance geometry $\longrightarrow$ Temporal organization $\longrightarrow$ Procrustes visualization

The important point is that reproducibility is not defined as equality of individual latent coordinates. Instead, the primary question is whether the *relationships among temporal states* are preserved across independently trained autoencoders.

1 Function `latent_columns()`

This function identifies all columns in the CSV file whose names begin with **z**, corresponding to latent coordinates such as

$$z_1, z_2, \dots, z_8.$$

It then sorts them numerically.

This is useful because an ordinary alphabetical ordering could potentially produce an ordering such as

$$z_1, z_{10}, z_2, z_3, \dots$$

instead of the desired numerical ordering.

The function therefore acts primarily as a data-safety utility.

2 Function `load_latents()`

This function loads a `latents.csv` file and verifies that:

- the file exists;
- a column named **step** is present;
- at least two latent dimensions are present;
- there are no duplicated temporal steps.

The rows are then sorted according to **step**.

At this stage no scientific comparison is performed. The function simply ensures that the latent trajectory is valid and consistently ordered.

3 Function pairwise_euclidean()

This is one of the fundamental operations of the analysis.

Suppose the latent trajectory contains 23 temporal states represented in an eight-dimensional latent space. The corresponding matrix has dimensions

$$X \in \mathbb{R}^{23 \times 8}.$$

Each row represents one temporal state,

$$\mathbf{z}(t_i) = (z_1(t_i), z_2(t_i), \dots, z_8(t_i)).$$

For every pair of temporal states t_i and t_j , the function calculates their Euclidean distance:

$$D_{ij} = \|\mathbf{z}(t_i) - \mathbf{z}(t_j)\|_2 = \sqrt{\sum_{k=1}^8 [z_k(t_i) - z_k(t_j)]^2}.$$

The result is a distance matrix

$$D \in \mathbb{R}^{23 \times 23}.$$

Schematically,

$$D = \begin{pmatrix} 0 & d_{12} & d_{13} & \cdots & d_{1,23} \\ d_{21} & 0 & d_{23} & \cdots & d_{2,23} \\ d_{31} & d_{32} & 0 & \cdots & d_{3,23} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ d_{23,1} & d_{23,2} & d_{23,3} & \cdots & 0 \end{pmatrix}.$$

This matrix describes the *internal relational geometry* of the latent trajectory.

Importantly, the analysis is no longer asking what the absolute coordinates of a state are. Instead, it asks how every state is positioned relative to every other state.

4 Function upper_triangle_values()

The Euclidean distance matrix is symmetric:

$$D_{ij} = D_{ji},$$

and its diagonal is zero:

$$D_{ii} = 0.$$

Therefore, only one triangular half of the matrix contains unique information.

The function retains the elements satisfying

$$i < j.$$

For $N = 23$ temporal states, the number of unique pairwise distances is

$$\binom{23}{2} = \frac{23 \times 22}{2} = 253.$$

Thus, each autoencoder trajectory is represented by a vector of 253 pairwise distances:

$$\mathbf{d}^{(A)} = (d_{12}^{(A)}, d_{13}^{(A)}, \dots, d_{22,23}^{(A)}).$$

This vector provides a compact representation of the complete pairwise distance geometry of autoencoder A .

5 Function `pearson_r()`

For two independently initialized autoencoders A and B , we obtain two distance vectors,

$$\mathbf{d}^{(A)} \quad \text{and} \quad \mathbf{d}^{(B)}.$$

The script calculates their Pearson correlation:

$$r_{AB} = \text{corr} \left(\mathbf{d}^{(A)}, \mathbf{d}^{(B)} \right).$$

More explicitly,

$$r_{AB} = \frac{\sum_i \left(d_i^{(A)} - \bar{d}^{(A)} \right) \left(d_i^{(B)} - \bar{d}^{(B)} \right)}{\sqrt{\sum_i \left(d_i^{(A)} - \bar{d}^{(A)} \right)^2} \sqrt{\sum_i \left(d_i^{(B)} - \bar{d}^{(B)} \right)^2}}.$$

A value such as

$$r_{AB} = 0.99$$

does *not* mean that the latent coordinates themselves are identical.

Rather, it means that pairs of temporal states that are relatively close or far apart in autoencoder A tend to have correspondingly close or far relationships in autoencoder B .

Thus, a high correlation indicates preservation of the relational geometry of the trajectory.

6 Functions `rankdata_average()` and `spearman_r()`

The script also computes the Spearman rank correlation.

Whereas Pearson correlation compares the numerical relationships among distances, Spearman correlation compares their ordering.

For example, suppose autoencoder A produces

$$d(A, B) < d(A, C) < d(A, D),$$

and autoencoder B also produces

$$d(A, B) < d(A, C) < d(A, D),$$

even though the actual numerical distances differ.

The Spearman correlation will remain high because the relative ranking of the distances is preserved.

Conceptually,

Pearson: Are the distances related approximately linearly?

whereas

Spearman: Is the relative ordering of the distances preserved?

The two measures therefore provide complementary descriptions of geometric reproducibility.

7 Function `validate_common_steps()`

Before comparing independently trained autoencoders, the script verifies that all trajectories contain:

- exactly the same temporal states;
- exactly the same latent dimensions.

For example, all runs should contain the same sequence

$$t_1, t_2, \dots, t_{23},$$

and the same latent coordinates

$$z_1, z_2, \dots, z_8.$$

If one trajectory were missing a temporal state, the correspondence

$$\mathbf{z}^{(A)}(t_i) \longleftrightarrow \mathbf{z}^{(B)}(t_i)$$

would no longer be guaranteed.

The function therefore prevents geometrically invalid comparisons.

8 Function `geometry_correlations()`

This is the central function of the analysis.

For every independently initialized autoencoder, the function performs the following transformation:

$$X^{(A)} \longrightarrow D^{(A)} \longrightarrow \mathbf{d}^{(A)}.$$

That is,

$$23 \times 8 \longrightarrow 23 \times 23 \longrightarrow 253.$$

For every pair of autoencoder initializations A and B , it then calculates

$$r_{AB} = \text{Pearson}(\mathbf{d}^{(A)}, \mathbf{d}^{(B)})$$

and

$$\rho_{AB} = \text{Spearman}(\mathbf{d}^{(A)}, \mathbf{d}^{(B)}).$$

If five independent initializations are analyzed, the number of unique autoencoder comparisons is

$$\binom{5}{2} = 10.$$

If six realizations are analyzed, as in the manuscript-specific robustness analysis containing the original Figure 5 realization plus five controlled retrainings, the number is

$$\binom{6}{2} = 15.$$

The central scientific question addressed by this function is therefore

Is the relational geometry of the complete latent trajectory reproducible across independent AE initializations

Crucially, this calculation is performed *before* any Procrustes alignment.

9 Function `temporal_metrics()`

The analysis now asks a different question.

Instead of comparing the complete geometry between autoencoders, it asks whether temporal neighbors remain local neighbors in latent space.

For each pair of states,

$$(t_i, t_j),$$

the script knows both their latent distance,

$$D_{ij} = d(\mathbf{z}(t_i), \mathbf{z}(t_j)),$$

and their temporal separation,

$$\Delta t_{ij} = |t_i - t_j|.$$

Pairs with

$$j - i = 1$$

are classified as adjacent temporal states.

Pairs satisfying

$$j - i > 1$$

are classified as non-adjacent states.

The script calculates

$$\bar{d}_{\text{adj}} = \frac{1}{N_{\text{adj}}} \sum_{\text{adjacent}} D_{ij}$$

and

$$\bar{d}_{\text{nonadj}} = \frac{1}{N_{\text{nonadj}}} \sum_{\text{nonadjacent}} D_{ij}.$$

It then forms the ratio

$$R_{\text{temporal}} = \frac{\bar{d}_{\text{adj}}}{\bar{d}_{\text{nonadj}}}.$$

A value

$$R_{\text{temporal}} \ll 1$$

means that consecutive temporal states are much closer to one another in latent space than arbitrary non-consecutive states.

For example,

$$R_{\text{temporal}} \approx 0.18$$

means that the average latent distance between adjacent temporal states is only about 18% of the average distance between non-adjacent temporal states.

The function also calculates the association between temporal separation

$$|t_i - t_j|$$

and latent-space distance

$$D_{ij}.$$

Thus it asks whether states that are increasingly separated in time also tend to become increasingly separated in latent space.

10 Function `orthogonal_procrustes_align()`

Consider two latent trajectories,

$$X, Y \in \mathbb{R}^{N \times p},$$

where N is the number of temporal states and p is the number of latent dimensions.

The trajectories are first centered:

$$X_c = X - \bar{X},$$

$$Y_c = Y - \bar{Y}.$$

The objective is then to find an orthogonal matrix R minimizing

$$\min_R \|X_c R - Y_c\|_F$$

subject to

$$R^\top R = I.$$

Because R is orthogonal, it can perform rotations and reflections.

It cannot perform scaling, stretching, compression, or nonlinear deformation.

The solution is obtained from the singular value decomposition

$$X_c^\top Y_c = U \Sigma V^\top.$$

The optimal orthogonal transformation is

$$R = UV^\top.$$

The aligned trajectory is therefore

$$X_{\text{aligned}} = X_c R.$$

For visualization, the reference centroid can subsequently be added back.

This operation addresses the fact that independently trained autoencoders may encode equivalent structures in differently oriented latent coordinate systems.

For example, two geometrically identical trajectories may differ only by a rotation:

$$X \neq Y,$$

while nevertheless satisfying

$$XR \approx Y$$

for an appropriate orthogonal transformation R .

Procrustes alignment removes this arbitrary orientation difference.

11 Relative Procrustes error

After alignment, the script calculates

$$E_{\text{Proc}} = \frac{\|X_c R - Y_c\|_F}{\|Y_c\|_F}.$$

This is a relative residual error.

If

$$E_{\text{Proc}} = 0,$$

then the two centered trajectories coincide exactly after an orthogonal rotation/reflection.

A non-zero value means that differences remain after alignment.

Therefore, the analysis does not claim that independently initialized autoencoders produce identical trajectories. Instead, Procrustes quantifies how closely their trajectories can be aligned without scaling or nonlinear deformation.

12 Determinant of the orthogonal transformation

For an orthogonal matrix,

$$R^T R = I,$$

the determinant satisfies

$$\det(R) \in \{-1, +1\}.$$

If

$$\det(R) = +1,$$

the transformation is a proper rotation.

If

$$\det(R) = -1,$$

the transformation includes a reflection.

This quantity is primarily diagnostic and is not the principal measure of latent-space reproducibility.

13 Function `procrustes_analysis()`

This function applies the orthogonal Procrustes procedure to every initialization relative to a chosen reference model.

For example,

reference = Seed 1.

Seed 1 has no privileged biological meaning. It simply defines a common coordinate system. For each other seed s , the analysis solves

$$\min_{R_s} \|X_{s,c}R_s - X_{\text{ref},c}\|_F.$$

After all trajectories have been aligned in the full latent space, the function derives a common two-dimensional visualization basis.

Importantly, the PCA basis is computed from the reference trajectory only.

If the centered reference trajectory is

$$X_{\text{ref},c},$$

its singular value decomposition is

$$X_{\text{ref},c} = U\Sigma V^\top.$$

The first two right-singular vectors define the common projection basis,

$$B = \begin{pmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{pmatrix}.$$

Thus the visualization coordinate system is defined once and then shared by all aligned trajectories.

14 Function `project_aligned_to_reference_pca()`

After Procrustes alignment in the complete latent space, every trajectory is projected onto the same two-dimensional reference-defined PCA basis.

For aligned trajectory X_s^{aligned} ,

$$P_s = \left(X_s^{\text{aligned}} - \bar{X}_{\text{ref}} \right) B.$$

Therefore,

$$\mathbb{R}^8 \longrightarrow \mathbb{R}^2$$

using the *same* transformation for every autoencoder.

This is important.

If an independent PCA were performed for every autoencoder, each trajectory would acquire its own two-dimensional coordinate system and the resulting plots would not be directly comparable.

15 Function `plot_geometry_heatmap()`

This function visualizes the pairwise geometry correlations.

Each cell represents

$$r_{AB} = \text{corr} \left(\mathbf{d}^{(A)}, \mathbf{d}^{(B)} \right).$$

For five independently initialized autoencoders, the visualization is a

$$5 \times 5$$

matrix.

The diagonal satisfies

$$r_{AA} = 1,$$

because each distance geometry is perfectly correlated with itself.

The off-diagonal elements quantify reproducibility between independent model initializations.

16 Function `plot_aligned_trajectories()`

This function visualizes the trajectories after the sequence

8D latent trajectory $\rightarrow$ orthogonal Procrustes alignment $\rightarrow$ common reference PCA projection $\rightarrow$ 2D plot

Only selected temporal endpoints are labeled to keep the figure readable.

This visualization is an aid to interpretation.

It is *not* the primary source of the statistical evidence for reproducibility.

17 Function `build_summary()`

This function collects the principal statistics for each autoencoder type.

These include:

- number of independent initializations;
- mean, minimum, and maximum Pearson geometry correlation;
- mean Spearman geometry correlation;
- mean adjacent/non-adjacent latent-distance ratio;
- temporal-separation versus latent-distance correlations;
- mean, minimum, and maximum relative Procrustes error.

It therefore converts the detailed pairwise analyses into a compact summary of initialization robustness.

18 Function `load_model_group()`

This function loads the latent trajectories associated with the requested model initialization seeds.

For example,

`global_seed_1, ..., global_seed_5`

or

`cluster_seed_1, ..., cluster_seed_5.`

It then calls the validation procedure to ensure that all trajectories are compatible.

This function contains no additional statistical analysis.

19 Function `parse_args()`

This function defines the command-line interface.

The most important arguments are:

- `-base_dir`: location of the latent trajectories;
- `-seeds`: model initialization seeds to compare;
- `-reference_seed`: reference used only for Procrustes alignment and the common visualization basis;
- `-outdir`: output directory.

An important distinction is that

`reference_seed` does not enter the distance-geometry correlation analysis.

Changing the Procrustes reference therefore does not change the primary pairwise geometry correlations.

20 Function `main()`

The `main()` function orchestrates the complete workflow.

For both the global-target and cluster-resolved autoencoders, it performs

load trajectories

followed by

distance-geometry analysis,

temporal-neighborhood analysis,

Procrustes alignment,

and

common 2D visualization.

It then exports the numerical tables, figures, projected coordinates, and summary statistics.

No additional mathematical transformation is introduced by `main()`; it acts as the “director” of the individual analysis components.

What does the script actually test?

In its most compact form, the script takes multiple latent representations of the same underlying temporal process obtained from independently initialized autoencoders and asks whether the relationships among temporal states remain stable despite changes in latent coordinates.

It evaluates this at three complementary levels.

1. Distance geometry

Distance geometry

asks:

Is the global relational structure of the latent trajectory preserved?

This is the primary reproducibility analysis.

2. Temporal neighborhood

Temporal neighborhood

asks:

Do temporally neighboring states remain local neighbors in latent space?

This characterizes local temporal organization.

3. Orthogonal Procrustes

Orthogonal Procrustes

asks:

If arbitrary differences in orientation and reflection of the latent coordinate systems are removed, without scaling or nonlinear deformation, do the trajectories exhibit comparable organization?

This provides an additional geometric control and a common visualization.

The logical order of the analysis

The scientific logic is deliberately

Distance geometry $\longrightarrow$ Temporal structure $\longrightarrow$ Procrustes visualization

and *not*

Procrustes alignment $\longrightarrow$ “the curves look similar”.

This distinction is essential.

The primary evidence for reproducibility is obtained from the complete latent-space geometry *before* alignment. Procrustes is subsequently used to remove arbitrary coordinate orientation and make the corresponding trajectory organization visually comparable.

Thus the central object being tested is not an individual latent coordinate such as z_1 or z_2 , but the system of relationships among the morphology-derived temporal states:

latent coordinates $\neq$ reproducible object

whereas the analysis asks whether

relational geometry of latent states $\approx$ reproducible structure.